sos Report

ruthvik

18-7-2026

Contents

1 Progress **3**

2 Things I Learned **3**

2.1 Time Complexity Analysis . . . 3

2.2 Selection Sort . . . 4

2.3 Bubble Sort . . . 4

2.4 Insertion Sort . . . 5

2.5 Insertion Sort Implementation . . . 5

2.6 Merge Sort . . . 5

2.7 Merge Sort Implementation . . . 6

2.8 Quick Sort . . . 6

2.9 Binary Search . . . 7

2.10 Structures . . . 8

2.11 Classes and Constructors (Basic Understanding) . . . 8

2.12 Two Pointer Technique . . . 8

2.13 Two Pointer Technique . . . 8

2.14 Sliding Window Technique . . . 9

2.15 Sliding Window Technique . . . 9

2.16 Sets and Unordered Sets (Basic Understanding) . . . 9

2.17 Stacks and Queues . . . 10

2.18 STL for Stack and Queue . . . 10
2.19 Deque, Circular Queue, and Simple Queue . . . 10
2.20 Singly Linked List . . . 10
2.21 Doubly Linked List . . . 11
2.22 Fast and Slow Pointer Technique . . . 12
2.23 Cycle Detection in Linked Lists . . . 12
2.24 Prefix Sum Arrays . . . 13
2.25 Difference Arrays . . . 13
2.26 Priority Queue . . . 14
2.27 Basic Hashing Concepts . . . 14
2.28 Hash Maps . . . 15
2.29 Binary Heaps . . . 15
2.30 Heapify Operation . . . 16
2.31 Heap Sort . . . 16
2.32 Priority Queues and Heaps . . . 17
2.33 Binary Trees . . . 18
2.34 Tree Traversals . . . 18
2.35 Binary Search Trees . . . 19
2.36 Insertion in a Binary Search Tree . . . 19
2.37 Recursion on Trees . . . 20
2.38 Recursion Tree Method . . . 20
2.39 Applications of Trees . . . 21
2.40 Introduction to Graphs . . . 21
2.41 Graph Representation . . . 21
2.42 Breadth First Search (BFS) . . . 22
2.43 Depth First Search (DFS) . . . 23
2.44 Topological Sorting . . . 23
2.45 Applications of Graph Algorithms . . . 24

3 Way I Did **25**

1 Progress

In this phase of my learning, I focused on building a strong base in core data structure concepts. At the beginning, most topics looked difficult and disconnected. But after studying them one by one and practicing the basic ideas, the topics started to make more sense.

I first worked on understanding how to think about time complexity, because that helps in comparing solutions and choosing better ones. After that, I learned the basic sorting algorithms like selection sort, bubble sort, and insertion sort. These helped me understand how arrays can be organized step by step.

Then I moved to slightly more advanced topics such as merge sort and quick sort. These made me see how recursion can be used to solve bigger problems by breaking them into smaller parts.

Later, I studied structures, classes, and constructors to understand how data can be grouped in a neat way. I also learned common problem-solving techniques such as two pointers and sliding window, which are very useful for array and string problems.

Finally, I studied sets and unordered sets, and then stacks, queues, and their STL versions. These topics helped me understand how to manage data in different orders depending on the problem.

2 Things I Learned

2.1 Time Complexity Analysis

Time complexity tells us how much time an algorithm may take as the input size grows. It is not about the exact clock time. It is about how the work increases when the input becomes larger.

A simple idea is this: if one method checks every element again and again, it will usually take more time than a method that reduces the work step by step. Big-O notation is used to describe this growth.

- **O(1)** means constant time, which means the work does not depend much on input size.
- **O(n)** means the time grows linearly with the input.
- **O(n^2)** means the time grows very fast, usually because of nested loops.

This concept is important because it helps us choose efficient solutions instead of slow ones.

2.2 Selection Sort

Selection sort is one of the simplest sorting algorithms. It works by finding the smallest element from the unsorted part and placing it at the beginning.

Imagine a row of numbers. You scan the whole row, pick the smallest one, and swap it into the first position. Then you repeat the same process for the remaining part.

It is easy to understand, but it is not very fast for large data. Its time complexity is usually **O(n^2)**.

```
int posMax(int* a, int n){
    int maxInd = 0;
    for(int i=0;i<n;i++) if(a[maxInd]<a[i]) maxInd=i;
    return maxInd;
}

void selsort(int* a, int n ){
    for (int i=n; i>1;i--){
        int temp = a[i-1];
        a[i-1] = a[posMax(a,i)];
        a[posMax(a,i)] = temp;
    }
}

void rselsort(int* a, int n){
    if(n==1) return;
    else{
        int temp = a[n-1];
        a[n-1] = a[posMax(a,n)];
        a[posMax(a,n)] = temp;
        rselsort(a,n-1);
    }
}
```

2.3 Bubble Sort

Bubble sort compares neighboring elements and swaps them if they are in the wrong order. After one full pass, the largest element moves to the end, like a bubble rising to the top.

This algorithm is simple and good for learning the basic idea of swapping and repeated passes. But like selection sort, it is slow for large inputs, with time complexity **O(n^2)**.

```
void swap(int &a, int &b){//& means we are passing the variables by
   reference, so any changes made to a and b inside the function will
   affect the original variables passed to the function.
    int temp = a;
    a = b;
```

```
    b = temp;
}

void bubsort(int* a,int n){
    //no of watys of selecting 2 are n-1 ways so
    for(int j=0;j<n-1;j++){//we have to do n-1 times
        for (int i=1; i<n-j; i++){//after j times the last j elements
           are in correct position so we can ignore them
        if (a[i-1]>a[i]){
            swap(a[i-1], a[i]);
        }
        else continue;
    }}
}
```

2.4 Insertion Sort

Insertion sort builds the sorted part gradually. It takes one element at a time and inserts it into the correct position among the already sorted elements.

This is similar to arranging playing cards in your hand. You pick up one new card and place it in the right spot.

It works well for small or nearly sorted arrays and is also easy to understand.

2.5 Insertion Sort Implementation

```
void inssort(int* a, int n){
    for (int i=0;i<n;i++){
        int ins =a[i];
        int j = i-1;
        while(j>=0&&ins<a[j]){
            a[j+1]=a[j];
            j--;//j can become negative
        }
    a[j+1]=ins;
    }
}
```

2.6 Merge Sort

Merge sort uses the divide-and-conquer approach. It splits the array into two halves, sorts each half separately, and then merges the two sorted halves.

This method is much more efficient than the simple quadratic sorting methods. Its time complexity is **O(n log n)**.

The main idea here is important: break a big problem into smaller pieces, solve them, and combine the answers.

2.7 Merge Sort Implementation

```
void mergeSortedParts(vector<int>& a, int l, int mid, int r) {
    vector<int> temp;
    int i = l, j = mid + 1;

    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else temp.push_back(a[j++]);
    }
    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, l, mid);
    mergeSort(a, mid + 1, r);
    mergeSortedParts(a, l, mid, r);
}
```

2.8 Quick Sort

Quick sort also uses divide and conquer. It selects a pivot element and rearranges the array so that smaller elements come before the pivot and larger elements come after it.

Then it repeats the same process on both sides.

Quick sort is often very fast in practice, and its average time complexity is **O(n log n)**. It is a key algorithm because it teaches partitioning and recursive thinking.

```
int partitionLomuto(vector<int>& a, int l, int r) {
    int pivot = a[r];
    int i = l - 1;
    for (int j = l; j < r; j++) {
        if (a[j] <= pivot) {
```

```
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[r]);
    return i + 1;
}

void quickSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int p = partitionLomuto(a, l, r);
    quickSort(a, l, p - 1);
    quickSort(a, p + 1, r);
}
```

2.9 Binary Search

Binary Search is an efficient searching algorithm that works only on sorted arrays.

Instead of checking every element one by one, Binary Search repeatedly divides the search space into two halves. It compares the target value with the middle element.

- If the target is equal to the middle element, the search is complete.
- If the target is smaller, we search in the left half.
- If the target is larger, we search in the right half.

Because the search space becomes half after every step, Binary Search is much faster than Linear Search for large datasets.

The time complexity of Binary Search is **O(log n)**.

A key requirement is that the data must be sorted before Binary Search can be applied.

```
int binarySearchIterative(const vector<int>& a, int key) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;   // safer than (l + r) / 2
        if (a[mid] == key) return mid;
        if (a[mid] < key) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}
```

2.10 Structures

A structure is a way to group related data together. Instead of storing many separate variables, we can keep them inside one unit.

For example, if we want to represent a student, we may store name, roll number, and marks together in a structure.

This makes code cleaner and easier to manage.

2.11 Classes and Constructors (Basic Understanding)

A class is similar to a structure, but it also helps us combine data and functions together. It is one of the main ideas of object-oriented programming.

A constructor is a special function that runs automatically when an object is created. Its job is to initialize the object with proper values.

This concept is useful because it helps make programs more organized, reusable, and easier to understand.

2.12 Two Pointer Technique

The two pointer technique uses two indexes to solve a problem more efficiently. These pointers may move in the same direction or from opposite ends, depending on the problem.

For example, one pointer may start at the left and the other at the right. By moving them carefully, we can reduce extra work.

This technique is very common in array and string problems, and it often helps reduce time complexity.

2.13 Two Pointer Technique

Finding whether a sorted array contains two numbers whose sum is equal to a target.

```
bool hasPair(vector<int>& arr, int target) {
    int left = 0;
    int right = arr.size() - 1;

    while(left < right) {
        int sum = arr[left] + arr[right];

        if(sum == target)
            return true;

        if(sum < target)
```

```
            left++;
        else
            right--;
    }

    return false;
}
```

2.14 Sliding Window Technique

Sliding window is used when we want to process a continuous part of an array or string. Instead of checking every possible range from scratch, we keep a window and move it step by step.

This saves time because we update the current answer instead of recalculating everything again.

It is useful in problems involving subarrays, substrings, sums, and counts.

2.15 Sliding Window Technique

Finding the maximum sum subarray of size k.

```
int maxSumSubarray(vector<int>& arr, int k) {
    int windowSum = 0;

    for(int i = 0; i < k; i++)
        windowSum += arr[i];

    int maxSum = windowSum;

    for(int i = k; i < arr.size(); i++) {
        windowSum += arr[i];
        windowSum -= arr[i - k];

        maxSum = max(maxSum, windowSum);
    }

    return maxSum;
}
```

2.16 Sets and Unordered Sets (Basic Understanding)

A set stores unique elements only. This means duplicates are automatically avoided.

A set keeps elements in sorted order, while an unordered set does not keep a fixed order but usually gives faster access.

These data structures are useful when we want to check whether something already exists, remove duplicates, or store distinct values efficiently.

2.17 Stacks and Queues

A stack follows the **LIFO** rule, which means *Last In, First Out*. The last element added is the first one removed.

A queue follows the **FIFO** rule, which means *First In, First Out*. The first element added is the first one removed.

These structures are very important because many real problems need controlled insertion and removal of data.

2.18 STL for Stack and Queue

STL gives ready-made implementations of stack and queue in C++. This saves time and reduces coding mistakes.

Instead of building everything from scratch, we can use the built-in library versions when the problem does not require a custom implementation.

2.19 Deque, Circular Queue, and Simple Queue

A deque is a double-ended queue. It allows insertion and deletion from both the front and the back.

A simple queue allows insertion at one end and deletion from the other end.

A circular queue is a more efficient queue design where the last position connects back to the first position. This helps in using space properly and avoiding waste.

These topics taught me how different data handling rules can be used depending on the type of problem.

2.20 Singly Linked List

A singly linked list is a collection of nodes connected together using pointers. Each node stores some data and the address of the next node in the sequence. Unlike arrays, linked lists do not require contiguous memory allocation, making them flexible when the size of the data changes during program execution.

One major advantage of a singly linked list is that insertion and deletion at the beginning of the list can be performed in constant time. However, accessing an element requires traversing the list from the beginning, resulting in linear time complexity.

The time complexity of insertion at the head is **O(1)**, while searching for an element takes **O(n)**.

```
struct Node{
    int data;
    Node* next;

    Node(int val){
        data = val;
        next = nullptr;
    }
};

void insertFront(Node* &head, int val){
    Node* temp = new Node(val);
    temp->next = head;
    head = temp;
}
```

2.21 Doubly Linked List

A doubly linked list is an extension of a singly linked list. In addition to storing a pointer to the next node, every node also stores a pointer to its previous node. This allows traversal in both forward and backward directions.

Because every node knows both its predecessor and successor, insertion and deletion become easier in many situations. The drawback is that each node requires additional memory to store the extra pointer.

Doubly linked lists are widely used in applications such as browser history, undo operations in editors, and navigation systems.

```
struct Node{
    int data;
    Node* prev;
    Node* next;

    Node(int val){
        data = val;
        prev = nullptr;
        next = nullptr;
    }
};
```

2.22 Fast and Slow Pointer Technique

The fast and slow pointer technique is a simple but powerful approach used mainly in linked list problems. Two pointers start from the beginning of the list. The slow pointer moves one node at a time while the fast pointer moves two nodes at a time.

Because the fast pointer travels twice as quickly, the slow pointer reaches the middle of the list when the fast pointer reaches the end. This idea can also be extended to solve several other linked list problems efficiently.

The technique requires only constant extra space and usually works in linear time.

```
Node* findMiddle(Node* head){

    Node* slow = head;
    Node* fast = head;

    while(fast && fast->next){
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}
```

2.23 Cycle Detection in Linked Lists

A linked list normally ends with a null pointer. However, due to programming errors or special problem settings, a node may point back to an earlier node, creating a cycle.

To detect such a cycle efficiently, Floyd's Cycle Detection Algorithm is used. It employs the fast and slow pointer technique. If the two pointers meet at any point while traversing the list, then a cycle exists.

This algorithm does not require any extra memory and detects cycles in linear time.

The time complexity is **O(n)** and the space complexity is **O(1)**.

```
bool hasCycle(Node* head){

    Node* slow = head;
    Node* fast = head;

    while(fast && fast->next){

        slow = slow->next;
        fast = fast->next->next;
```

```
        if(slow == fast)
            return true;
    }

    return false;
}
```

2.24 Prefix Sum Arrays

Prefix sums provide an efficient method for answering multiple range sum queries. Instead of computing the sum of every range separately, cumulative sums are calculated once and stored in another array.

After preprocessing, the sum of any subarray can be obtained in constant time using simple subtraction. This technique is especially useful when many queries are performed on the same array.

Constructing the prefix array requires **O(n)** time, while each range query requires only **O(1)** time.

```
vector<int> prefix(n);

prefix[0] = a[0];

for(int i = 1; i < n; i++)
    prefix[i] = prefix[i-1] + a[i];

int rangeSum(int l, int r){

    if(l == 0)
        return prefix[r];

    return prefix[r] - prefix[l-1];
}
```

2.25 Difference Arrays

Difference arrays are useful when multiple range update operations need to be performed efficiently. Instead of updating every element inside a range individually, only the boundaries of the range are modified.

After all updates are completed, taking the prefix sum of the difference array reconstructs the final updated array. This significantly reduces the overall time complexity when many updates are involved.

This technique is commonly used in competitive programming and interval update problems.

```
vector<int> diff(n + 1, 0);

diff[l] += value;

if(r + 1 < n)
    diff[r + 1] -= value;

for(int i = 1; i < n; i++)
    diff[i] += diff[i - 1];
```

2.26 Priority Queue

A priority queue is a special data structure in which every element has a priority associated with it. The element with the highest priority is removed before the others, regardless of the order in which elements were inserted.

In C++, the Standard Template Library implements priority queues using heaps. By default, the largest element always remains at the top of the priority queue.

Insertion and deletion both require **O(log n)** time, while accessing the top element takes **O(1)** time.

Priority queues are widely used in graph algorithms such as Dijkstra's Algorithm, scheduling problems, and simulation tasks.

```
priority_queue<int> pq;

pq.push(10);
pq.push(5);
pq.push(25);

cout << pq.top() << endl;

pq.pop();

cout << pq.top() << endl;
```

2.27 Basic Hashing Concepts

Hashing is a technique used to store and retrieve data efficiently. Instead of searching through every element one by one, a hash function converts a key into an index where the data can be stored.

The main objective of hashing is to achieve very fast lookup operations. In most practical cases, searching, insertion, and deletion can be performed in constant average time.

One important concept in hashing is collision, which occurs when two different keys produce the same hash value. Different collision resolution techniques such as chaining and open addressing are used to handle such situations.

The average time complexity of searching, insertion, and deletion is **O(1)**, while the worst case can become **O(n)**.

2.28 Hash Maps

A hash map stores data in the form of key-value pairs. Unlike arrays where elements are accessed using integer indices, hash maps allow any suitable data type to be used as the key.

The C++ Standard Template Library provides the `unordered_map` container, which internally uses hashing. It allows efficient insertion, lookup, and deletion of elements.

Hash maps are commonly used for counting frequencies, checking whether an element exists, storing mappings, and solving many competitive programming problems.

```
#include <unordered_map>
using namespace std;

unordered_map<string,int> marks;

marks["Alice"] = 95;
marks["Bob"] = 88;

cout << marks["Alice"] << endl;

if(marks.count("Bob"))
    cout << "Found";
```

2.29 Binary Heaps

A heap is a complete binary tree that satisfies the heap property.

In a max heap, every parent node is greater than or equal to its children. In a min heap, every parent node is smaller than or equal to its children.

Because heaps are complete binary trees, they are usually implemented using arrays instead of linked structures. This makes the implementation simpler and more memory efficient.

Insertion and deletion both require **O(log n)** time, while finding the maximum or minimum element requires only **O(1)** time.

```
vector<int> heap = {50,40,35,20,10,30};
```

```
int leftChild(int i){
    return 2*i+1;
}

int rightChild(int i){
    return 2*i+2;
}

int parent(int i){
    return (i-1)/2;
}
```

2.30 Heapify Operation

Heapify is the fundamental operation used to maintain the heap property after inserting or deleting elements.

If a node violates the heap property, it is repeatedly swapped with the appropriate child until the entire subtree satisfies the heap condition.

Heapify forms the basis for many heap algorithms, including heap construction and heap sort.

The time complexity of the Heapify operation is **O(log n)**.

```
void heapify(vector<int>& a,int n,int i){

    int largest=i;
    int left=2*i+1;
    int right=2*i+2;

    if(left<n && a[left]>a[largest])
        largest=left;

    if(right<n && a[right]>a[largest])
        largest=right;

    if(largest!=i){
        swap(a[i],a[largest]);
        heapify(a,n,largest);
    }
}
```

2.31 Heap Sort

Heap Sort is a comparison-based sorting algorithm that uses a binary heap.

The algorithm first converts the array into a max heap. The largest element is then placed at the end of the array by swapping it with the root. The heap size is reduced, and the heap property is restored. This process continues until all elements are sorted.

Heap Sort has a guaranteed time complexity of **O(n log n)** in the best, average, and worst cases. Unlike Merge Sort, it does not require extra memory for another array.

```
void heapSort(vector<int>& a){

    int n=a.size();

    for(int i=n/2-1;i>=0;i--)
        heapify(a,n,i);

    for(int i=n-1;i>0;i--){

        swap(a[0],a[i]);

        heapify(a,i,0);
    }
}
```

2.32 Priority Queues and Heaps

After studying heaps, I understood how priority queues are implemented internally.

A priority queue maintains the highest (or lowest) priority element at the top. Every insertion or deletion automatically adjusts the heap so that the heap property is preserved.

This makes priority queues very useful in algorithms where the largest or smallest element needs to be accessed repeatedly, such as Dijkstra's shortest path algorithm, Prim's algorithm, scheduling problems, and simulation systems.

```
priority_queue<int> pq;

pq.push(12);
pq.push(40);
pq.push(7);
pq.push(25);

while(!pq.empty()){
    cout << pq.top() << " ";
    pq.pop();
}
```

2.33 Binary Trees

A binary tree is a hierarchical data structure in which every node has at most two children, called the left child and the right child. The first node is called the root, and nodes without children are known as leaf nodes.

Binary trees provide an efficient way to represent hierarchical information. Many algorithms can be naturally expressed using recursion because every subtree is itself a binary tree.

Binary trees are commonly used in expression evaluation, syntax trees, Huffman coding, and many searching algorithms.

```
struct Node{

    int data;
    Node* left;
    Node* right;

    Node(int val){
        data = val;
        left = nullptr;
        right = nullptr;
    }
};
```

2.34 Tree Traversals

Tree traversal refers to visiting every node of a tree exactly once in a particular order. The three depth-first traversals are Preorder, Inorder, and Postorder.

Preorder visits the root before its children, Inorder visits the left subtree before the root, and Postorder visits the children before the root. Each traversal is useful for different applications.

These traversals can be implemented naturally using recursion.

The time complexity of each traversal is $\mathbf{O(n)}$.

```
void inorder(Node* root){

    if(root == nullptr)
        return;

    inorder(root->left);

    cout << root->data << " ";

    inorder(root->right);
}
```

2.35 Binary Search Trees

A Binary Search Tree (BST) is a special type of binary tree that satisfies an ordering property.

For every node, all values in the left subtree are smaller than the node, while all values in the right subtree are greater than the node. This property allows searching, insertion, and deletion to be performed efficiently.

In a balanced Binary Search Tree, searching requires only **O(log n)** time. However, if the tree becomes skewed, the worst-case complexity becomes **O(n)**.

Binary Search Trees form the foundation for several advanced data structures such as AVL Trees and Red-Black Trees.

```
Node* search(Node* root,int key){

    if(root == nullptr || root->data == key)
        return root;

    if(key < root->data)
        return search(root->left,key);

    return search(root->right,key);
}
```

2.36 Insertion in a Binary Search Tree

Insertion in a Binary Search Tree follows the BST property. Starting from the root, the new value is compared with the current node. If it is smaller, the algorithm moves to the left subtree; otherwise, it moves to the right subtree. This process continues until an empty position is found.

Because only one path from the root to a leaf is visited, insertion is efficient for balanced trees.

```
Node* insert(Node* root,int key){

    if(root == nullptr)
        return new Node(key);

    if(key < root->data)
        root->left = insert(root->left,key);
    else
        root->right = insert(root->right,key);

    return root;
}
```

2.37 Recursion on Trees

Trees are naturally recursive because every subtree is itself a tree. This makes recursive algorithms both simple and intuitive.

Many common operations such as calculating the height of a tree, counting nodes, finding leaf nodes, and checking tree properties are implemented recursively.

Understanding recursion on trees also helped me improve my understanding of recursive thinking developed during earlier weeks.

```
int height(Node* root){

    if(root == nullptr)
        return 0;

    return 1 + max(height(root->left),
                   height(root->right));
}
```

2.38 Recursion Tree Method

Apart from recursion on data structures, I also studied the Recursion Tree Method from CLRS for analyzing recursive algorithms.

The main idea is to represent every recursive call as a node in a tree. The total running time is obtained by adding the work done at every level of the recursion tree.

This method provides an intuitive way of understanding recurrence relations before solving them mathematically. It is especially useful for divide-and-conquer algorithms such as Merge Sort and Quick Sort.

For example, the recurrence

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

forms a recursion tree where every level performs approximately n units of work. Since there are $\log n$ levels, the overall complexity becomes

$$T(n) = O(n \log n)$$

Studying this method helped me understand why many divide-and-conquer algorithms have logarithmic depth and efficient running times.

2.39 Applications of Trees

Trees are one of the most widely used data structures in computer science.

Some important applications include:

- File system organization
- Database indexing
- Expression trees in compilers
- XML and HTML document representation
- Decision trees in Machine Learning
- Binary Search Trees for efficient searching

Learning these applications helped me understand why trees are considered one of the fundamental data structures.

2.40 Introduction to Graphs

A graph is a collection of vertices (nodes) and edges that connect pairs of vertices. Graphs can be either directed or undirected. In a directed graph, every edge has a direction, whereas in an undirected graph, edges can be traversed in both directions.

Graphs may also be weighted, where every edge stores an associated cost or distance, or unweighted, where all edges are treated equally.

Graphs provide a flexible way to represent relationships between different entities and are one of the most widely used data structures in computer science.

2.41 Graph Representation

Before applying graph algorithms, it is important to choose an appropriate representation.

The two common methods are the adjacency matrix and the adjacency list. An adjacency matrix stores the connectivity information in a two-dimensional matrix, whereas an adjacency list stores the neighbors of every vertex in a list.

For sparse graphs, adjacency lists are generally preferred because they require much less memory.

The time required to build an adjacency list is $\mathbf{O(V + E)}$, where V is the number of vertices and E is the number of edges.

```
int n = 5;
```

```
vector<vector<int>> adj(n);

adj[0].push_back(1);
adj[0].push_back(2);

adj[1].push_back(3);
adj[2].push_back(4);
```

2.42 Breadth First Search (BFS)

Breadth First Search (BFS) is a graph traversal algorithm that explores vertices level by level. It starts from a source vertex and visits all of its neighboring vertices before moving to the next level.

BFS uses a queue to maintain the order in which vertices should be explored. It guarantees the shortest path in an unweighted graph because vertices are visited according to their distance from the source.

The time complexity of BFS is $\mathbf{O(V + E)}$.

```
void bfs(int start,
         vector<vector<int>>& adj){

    vector<bool> vis(adj.size(),false);

    queue<int> q;

    vis[start]=true;
    q.push(start);

    while(!q.empty()){

        int node=q.front();
        q.pop();

        cout<<node<<" ";

        for(int next:adj[node]){

            if(!vis[next]){

                vis[next]=true;
                q.push(next);
            }
        }
    }
}
```

2.43 Depth First Search (DFS)

Depth First Search (DFS) explores one path as deeply as possible before backtracking. It is naturally implemented using recursion, although it can also be implemented using an explicit stack.

DFS is useful for many graph problems such as connected components, cycle detection, topological sorting, and path finding.

Like BFS, the overall time complexity of DFS is $\mathbf{O(V + E)}$.

```
void dfs(int node,
         vector<vector<int>>& adj,
         vector<bool>& vis){

    vis[node]=true;

    cout<<node<<" ";

    for(int next:adj[node]){

        if(!vis[next])
            dfs(next,adj,vis);
    }
}
```

2.44 Topological Sorting

Topological Sorting is applicable only to Directed Acyclic Graphs (DAGs). It produces a linear ordering of vertices such that for every directed edge $u \to v$, vertex u appears before vertex v.

This ordering is useful whenever certain tasks must be completed before others. Common applications include scheduling jobs, resolving software dependencies, and determining compilation order.

One common algorithm for topological sorting is Kahn's Algorithm, which repeatedly removes vertices having zero indegree.

The overall time complexity is $\mathbf{O(V + E)}$.

```
vector<int> topoSort(int n,
                     vector<vector<int>>& adj){

    vector<int> indegree(n,0);

    for(int u=0;u<n;u++)
        for(int v:adj[u])
            indegree[v]++;
```

```
    queue<int> q;

    for(int i=0;i<n;i++)
        if(indegree[i]==0)
            q.push(i);

    vector<int> order;

    while(!q.empty()){

        int node=q.front();
        q.pop();

        order.push_back(node);

        for(int next:adj[node]){

            indegree[next]--;

            if(indegree[next]==0)
                q.push(next);
        }
    }

    return order;
}
```

2.45 Applications of Graph Algorithms

Graph algorithms are used in many real-world systems and software applications.

Some important applications include:

- GPS and shortest path navigation
- Social network analysis
- Computer and communication networks
- Dependency resolution in software packages
- Web page crawling by search engines
- Course scheduling and project planning

Understanding these applications helped me appreciate why graphs are one of the most important topics in computer science.

3 Way I Did

1. First, I read the concept in a basic way so I could understand the main idea.
2. Then, I wrote small examples by hand to see how the algorithm or data structure behaves.
3. After that, I practiced coding did few easy and very easy problems to understand the concept better
4. I compared similar topics,
5. I also revisited topics like time complexity again and again.

This method helped me move from only reading theory to actually understanding how the concepts work in practice.